

Measuring the time efficiency of development and the execution speed of the code between C and Python through controlled tasks

David Gaby Sahala Sinambela
Artificial Intelligence
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
david.sinambela@binus.ac.id

Fiko Alexie van Houten
Artificial Intelligence
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
fiko.houten@binus.ac.id

Richtjhie Hartawan Agusta
Artificial Intelligence
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
richtjhie.agusta@binus.ac.id

Said Achmad
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
said.achmad@binus.edu

Maulin Nasari
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
maulin.nasari@binus.edu

Abstract—This research explores the debate between C and Python. C being the low-end programming language is known for its fast execution while being hard to develop. Whereas Python being the high-end programming language is known for the complete opposite, fast at development and slow at execution.

I. INTRODUCTION

The selection of a programming language inherently involves a trade-off between development efficiency and execution speed. In practice, this often means a trade-off: low-level languages like C deliver superior execution speed and fine-grained hardware control, whereas high-level languages like Python enable much faster coding and prototyping [1], [2]. For example, one study notes that C code “runs under a compiler” and thus “code execution is faster” than in Python, whose interpreter and garbage collector impose additional runtime overhead [1], [2]. Conversely, Python programs tend to be far shorter and easier to write as Rohit Kumar and Chander observe, “Python programs are easier to learn, write, and read,” and the development process in Python is generally much faster than in C [1], [3], [5]. So, the question arises, does the hidden cost of C’s manual memory management and harder syntaxes make C less superior to Python’s convenience that sacrifices extra CPU power and energy cost? [1]-[4], [18].

The cause of these differences is the language’s execution models. The source code of C is translated directly into machine code before running, which allows the execution to run directly on the CPU with minimal overhead [2], [4], [14], [20]. In contrast, Python’s source code is translated into bytecodes, which then a Virtual Machine executes at runtime, making it slower because it incurs overhead from the interpretation layer and runtime checks. These abstractions make Python very, very, developer-friendly [5], [23] but there’s a significant increase in execution latency [1], [2], [19].

We conduct this study, primarily, to find the correlation between development effort and computational output to provide a framework for both C and Python. The main

objective is to evaluate whether the reduced development time offered by Python justifies the sacrifice from execution speed [1], [2], [6], [9]. To achieve this, the research aims to make a methodology using controlled tasks, where there are identical challenges of algorithm programming from simple data processing to complex resource-intensive computations for both C and Python [2], [6], [9], [12]. The goal of this paper is to figure out a data framework so that engineers could then decide whether to use Python’s fast development with the slower execution speed, or C’s insanely fast runtime justifies the slower development efforts.

This research primary contribution is in the design of a controlled, reproducible experiment that can simultaneously evaluate the development time to research and write the code, but also the runtime efficiency, within the same tasks and the same conditions [6], [12], [17]. By using these two dimensions, which could be seen in prior works as separate concerns, this research hopes to establish a coherent empirical reference point for programming language decisions. Furthermore, because the methodology is documentable but also reproducible, the framework could be used for other researchers that may apply different language pairs or different hardware environments [6], [12].

II. LITERATURE REVIEW

Linkage between programming language choice and productivity of developers has been a subject of extensive scholarly investigation for a few decades now; however, the main conclusions made have not changed very much over time. Compilers result in increased program speed, and high-level languages make coding quicker. The difference in performance is real and context-related. At the same time, research which evaluates the two aspects at once and makes the comparison possible in comparable metrics is relatively rare.

When it comes to performance, C and C++ clearly outperform any other languages on virtually all of the tested domains. According to Kumar, the competitive advantage of the two programming languages can be viewed through the lens of their lack of the virtual machine, interpreter, and

garbage collector, which consume processor resources [7]. Also, according to Patel and Kishorhbai, C is still relevant in today's programming due to its ability to give programmer more control over memory and processor usage as compared to interpreted languages; such advantage is not sentimental, but rather based on the way the language maps to hardware [14], [20].

Indeed, empirical studies in benchmark testing always confirm this tendency. The work of Rysak compares C and Python in different classical algorithms and proves that C was better in terms of execution time in all cases and the gap between languages grew larger depending on complexity of the task [19]. It is worth noting that the study carried out in 2024 by Zhang on topological sorting algorithms is highly relevant to the current experiment because it proved the dominance of C++ which uses the same way of execution as C and stated that it was highly preferable compared to Python in nearly all cases while calling for further research on development costs for faster programs [12]. Shoaib confirmed this tendency for a wider variety of classical algorithms and found that compiled languages provide higher execution performance independently of the chosen algorithm [28].

It is important to note at the beginning that the most significant feature of existing studies in this field is their focus on development effort and runtime performance as separate components. There are a lot of works which provide quantitative evaluation of performance during algorithm implementation and energy efficiency in execution and there are a lot of works which provide qualitative assessment of developer productivity [6], [17]. The reason why there are few works which measure the trade-off between development effort and runtime performance is the lack of empirical research which evaluates how much effort developers save by using Python compared with how much time computers lose because of slower execution [6], [17].

Inspired by prior algorithmic comparisons [6], [17]. Almost all of the reference papers in this paper are comparable in two programming languages. The research of Tao and Zhang et al. on programming assignment time efficiency in terms of Controlled Task methodology is the most relevant to our research among existing studies. It measures time spent and efforts put into completing algorithmic assignments in different languages and shows that users of high-level language finish their tasks faster while the used language provides a higher benchmark of computational performance but less efficient solutions [9], [17], [18], [19].

III. METHODOLOGY

This is a quantitative experiment using Controlled Task methods. The core principle is to measure the trade-off between writing code and running code at the same time on the same problems and under the same conditions [6], [9], [17]. The research method can be summarized as follows:

The independent variable in this study is the programming language: C and Python. The dependent variables are development (measured in minutes), Lines of Code, and execution speed of both solutions (C and Python). All other factors such as the hardware used for benchmarking, the problem specifications, and test inputs

are held constant as control variables in the experiment [7], [9], [12]

Volunteer programmers as experimental subjects, so the development time data reflects actual programmer behavior rather than one researcher's performance. Experimental subjects volunteer programmers must have prior experience with both C and Python programming. Volunteers are informed of the study's purpose and provide their consent to participate. All data collected from volunteers is anonymized and stored securely; no personally identifiable information is reported in the study's results [17].

To make sure the comparison is done fairly and generalizable across different types of algorithmic challenges, the experiment is structured into three difficulty levels each with three tasks:

- 1) *Level 1 (Easy)*: Basic syntax, simple logic
- 2) *Level 2 (Medium)*: Intermediate algorithms, basic data structures
- 3) *Level 3 (Hard)*: Complex logic, algorithms, or computationally intensive calculations

Each volunteer solves all nine problems once in C and once in Python. Volunteers are allowed to use tools such as w3schools and GeekForGeeks to help them develop the code; volunteers are not allowed to use AI [17].

1) *Input Data Generation:*

Large-scale data will be made using AI. For example, both the C and Python algorithms will be using generated data as the input to ensure fairness in the comparison. [1], [9], [12].

2) *Development Metrics:*

The two main metrics are development time and Lines of Code (LoC). Development time will be measured in seconds using a stopwatch starting from a blank page to the finished solutions. The LoC will be measured by lines of code excluding comments.

3) *Execution Metric:*

The main metric for execution is execution time that is measured in milliseconds using language native timing functions. Each program will then be executed 5 times, and the average time will be used to reduce the variance of the result [14], [17].

4) *Hardware Environment*

All tests are conducted on a single machine to ensure consistency throughout the study [7], [11]. The benchmark computer was equipped with an AMD Ryzen 7 4800H, 16GB RAM, and Windows 11 Home 64-bit OS. C programs were compiled using GCC version 15.1.0 with default optimization settings and Python programs were executed using Python 3.13.3.

This experiment uses a structured six step pipeline. Firstly, the volunteer receives a problem statement. Secondly, the timer starts and they begin coding from a blank file. Thirdly, the timer stops when the solution for each problem (one solution for each problem per language) is finished. Fourthly, development time is recorded and LoC is counted. Fifthly, the submitted code is collected and sent to the author. Sixth, after all sessions are complete, the team

runs each submitted program on the benchmark machine and records execution time and memory usage.

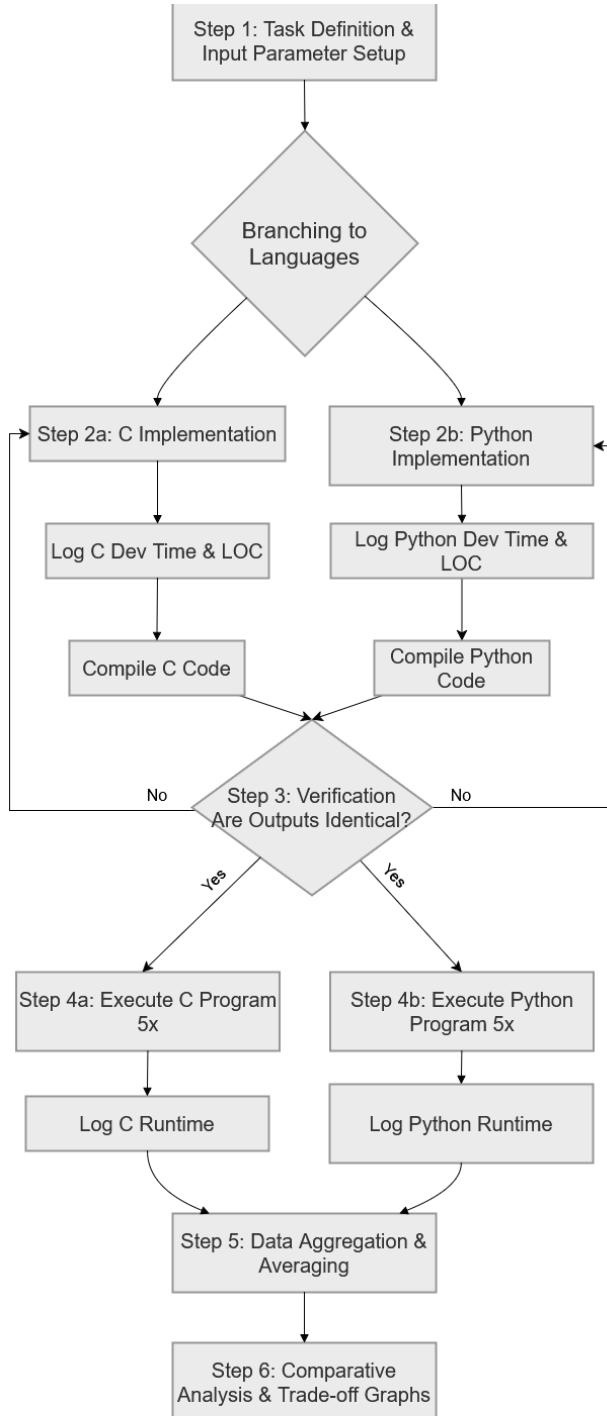


Fig. 1: Experimental Workflow Pipeline

This is a quantitative experiment using Controlled Task methods. The core principle is to measure the trade-off between writing code and running code at the same time on the same problems and under the same conditions [6], [9], [17]. The research method can be summarized as follows:

The independent variable in this study is the programming language: C and Python. The dependent variables are development (measured in minutes), Lines of Code, and execution speed of both solutions (C and Python). All other factors such as the hardware used for benchmarking, the problem specifications, and test inputs

are held constant as control variables in the experiment [7], [9], [12]

Volunteer programmers as experimental subjects, so the development time data reflects actual programmer behavior rather than one researcher's performance. Experimental subjects volunteer programmers must have prior experience with both C and Python programming. Volunteers are informed of the study's purpose and provide their consent to participate. All data collected from volunteers is anonymized and stored securely; no personally identifiable information is reported in the study's results [17].

To ensure the comparison is fair and generalizable across different types of computational challenges, the experiment is structured around nine problems organized into three difficulty levels:

- 4) *Level 1 (Easy)*: Basic syntax, simple logic
- 5) *Level 2 (Medium)*: Intermediate algorithms, basic data structures
- 6) *Level 3 (Hard)*: Complex logic, algorithms, or computationally intensive calculations

Each volunteer solves all nine problems once in C and once in Python. Volunteers are allowed to use tools such as w3schools and GeekForGeeks to help them develop the code; volunteers are not allowed to use AI [17].

5) Input Data Generation:

Large-scale data will be made using AI. For example, both the C and Python algorithms will be using generated data as the input to ensure fairness in the comparison. [1], [9], [12].

6) Development Metrics:

The two main metrics are development time and Lines of Code (LoC). Development time will be measured in seconds using a stopwatch starting from a blank page to the finished solutions. The LoC will be measured by lines of code excluding comments.

7) Execution Metric:

The main metric for execution is execution time that is measured in milliseconds using language native timing functions. Each program will then be executed 5 times, and the average time will be used to reduce the variance of the result [14], [17].

8) Hardware Environment

All tests are conducted on a single machine to ensure consistency throughout the study [7], [11]. The benchmark computer was equipped with an AMD Ryzen 7 4800H, 16GB RAM, and Windows 11 Home 64-bit OS. C programs were compiled using GCC version 15.1.0 with default optimization settings and Python programs were executed using Python 3.13.3.

This experiment uses a structured six step pipeline. Firstly, the volunteer receives a problem statement. Secondly, the timer starts and they begin coding from a blank file. Thirdly, the timer stops when the solution for each problem (one solution for each problem per language) is finished. Fourthly, development time is recorded and LoC is counted. Fifthly, the submitted code is collected and sent to the author. Sixth, after all sessions are complete, the team

runs each submitted program on the benchmark machine and records execution time and memory usage.

1]

IV. RESULT AND DISCUSSION

This chapter presents our empirical examinations from our controlled comparison between C and Python from 9 different algorithmic problems with varying difficulty. We have conjured 324 observations from 20 participants among which are undergraduate computer scientists from Binus Kemangisan that have practical experience with both C and Python.

As previously described, the problems are divided into 3 varying difficulty category, first being easy (E1, E2, and E3) which includes basic tasks and logics, there's medium (M1, M2, and M3) which includes intermediate algorithms and data structures, and finally we have hard (H1, H2, and H3) which includes complex logics and computationally hard tasks to run.

The primary variables are development time (seconds) using a stopwatch from a blank file to a finished algorithm, LoC which are the executable lines excluding comments, and finally the execution speed (ms) which is the runtime that is averaged out from 5 repeated runs. We also decided for the execution speed to use two representative problems H1 (sorting) and H2 (matrix transpose), each tested with three randomly generated input sizes. For H1 we have 100, 500, and 1000 random integers and for H2 we have 100 x 100, 500 x 500, and 1000 x 1000 random matrices (both of which are generated by Claude AI)

One of the very first things we found was that across 20 participants the gap in development time between Python and C seems to appear very significant.

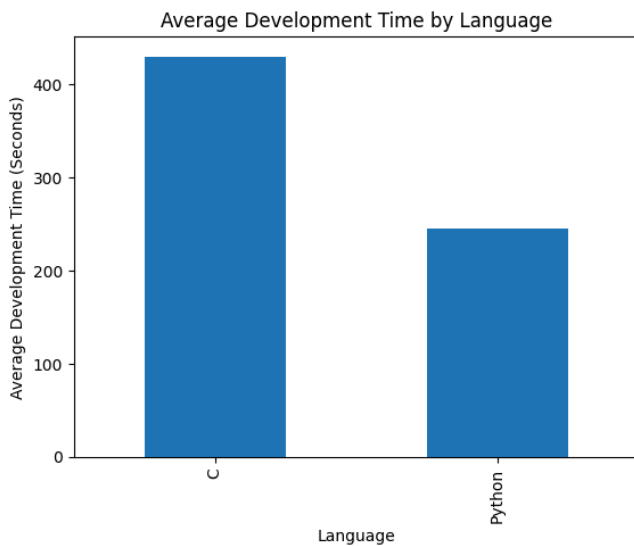


Fig. 2: Average Development Time by Language

Figure 2 shows the average development time required to complete the programming tasks in each language. Across all difficulties it is shown that participants could write the code almost 1.8x faster in Python compared to C. These results align very well with prior research, which states that due to Python's higher level syntax and built-in data structure, it reduces cognitive load which allows the

participants to implement the algorithm much easier [3], [17]. We could continue to see the significance of difficulty in Figure 3.

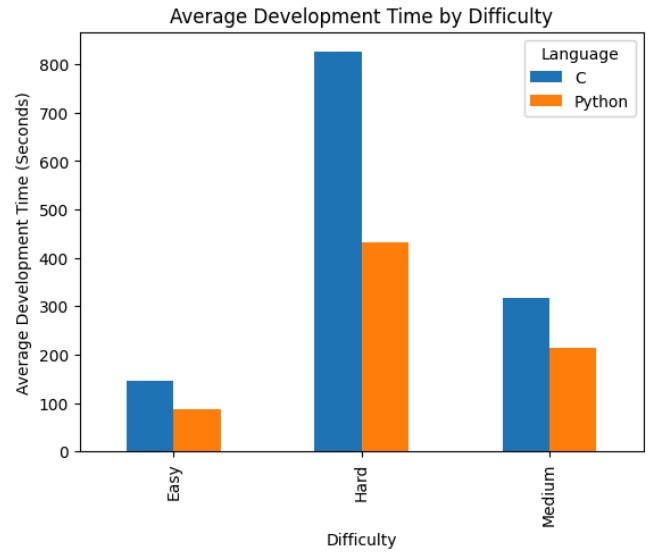


Fig. 3: Average Development Time by Difficulty

The impact of difficulty starts to be more pronounced as the harder tasks appear. This could indicate that as the task complexity increases the productivity differences become more apparent between the two languages. Figure 4 will show how each task affects the development time.

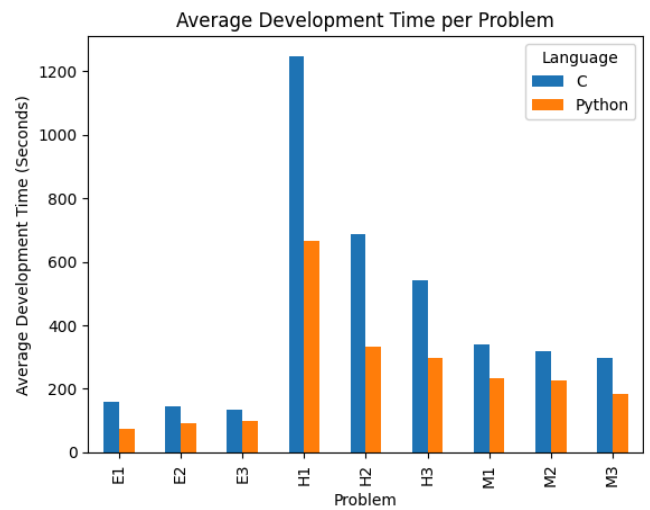


Fig. 4: Average Development Time per Problem

Figure 4 shows that there is a very significant difference in each of the problems, primarily H1, in which there is a spike that is shown on the graph that took participants almost twice as long in C as they did in Python. This reduction in development time could also partially explain the length of code written. Figure 5 will show that exactly.

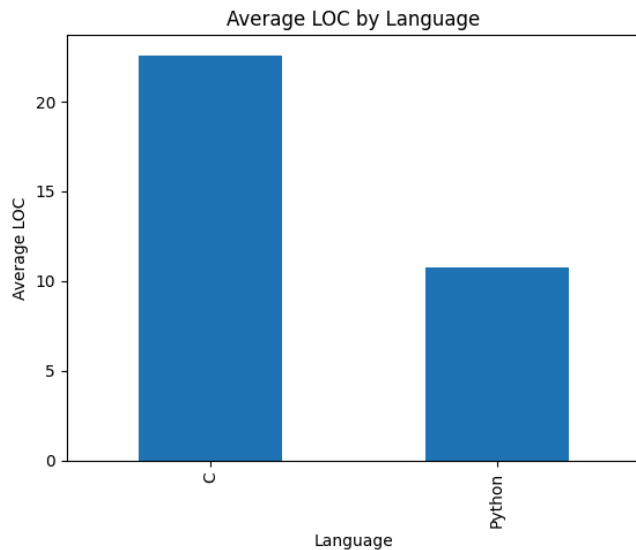


Fig. 5: Average LoC by Language

Figure 5 shows the average execution line the participants wrote their code in by language. This graph indicates that it took participants almost 2x as many lines of code in C than in Python. This reflects the fundamental different design of both C and Python. C is a language that requires explicit type declaration for all variables, manual memory allocation and delocation, verbose loop syntaxes, and explicit casting and conversion. Python has dynamic typing, built in lists, dictionary, Python also handles string and memory automatically. Now LoC may not influence development time directly but both stem from the fact that C is a low-level language that requires more explicit instructions to obtain the same result.

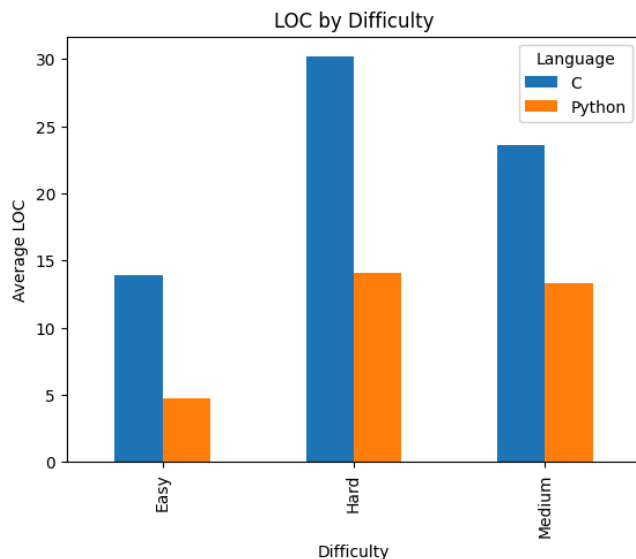


Fig. 6: Average LoC by Difficulty

Figure 6 shows us that the LoC remains constant in three of the difficulty varieties. C takes twice as many lines than it does with Python although Python takes it a step further in the easier category having more than 3x less lines needed than C. This reflects the C's more verbose syntax, including declaration of variables and memory handling that takes up most of the lines more than the actual algorithm. Figure 7 will show each of the LoC by tasks.

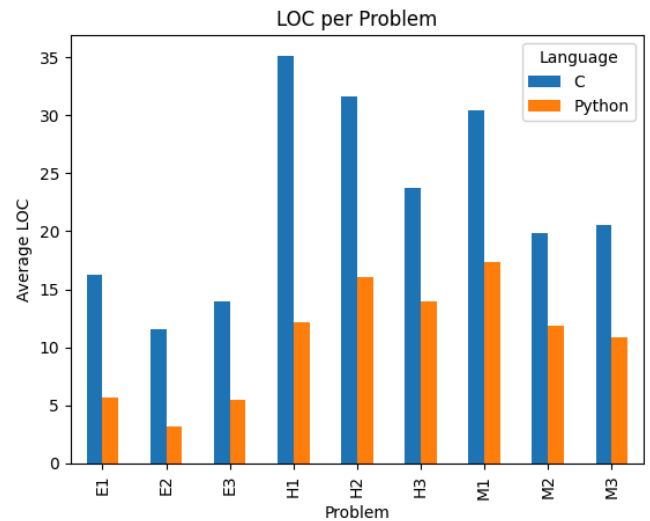


Fig. 7: Average LoC per Problem

Figure 7 now shows the difference of LoC in each question. We could see the same exact trend with development time on the H1 that seems very significant, other than this the differences don't really differ that much. Python takes less than half lines of code than C does.

Now although Python has shown dominance regarding development time and LoC, the same cannot be said with execution speed, as decided the two representative tasks will be H1 and H2. These two tasks alone could demonstrate both strengths and weaknesses of each language.

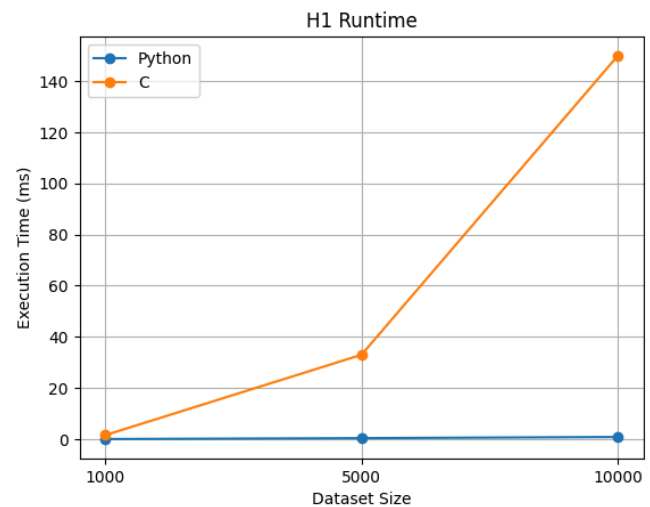


Fig. 8: H1 Runtime

Interestingly, H1 produced results that are pretty opposing to the general expectation that C would perform better than Python. But here in figure 8 we could see that Python does exceptionally better than C. This is however not really a head to head battle between each language, more so a showcase of the superior convenience of Python. To elaborate, most of the participants while coding in C would prefer to use the Bubble Sort which is an easy code to implement though not so fast as only having an $O(n^2)$ worst case. Whereas when coding in Python, participants use the built in `sort()` function in Python, which uses Timsort and is way faster than bubble sort having an $O(n \log n)$ worst case.

This H1 runtime doesn't display C's strength in execution but rather Python's strength in convenience. Python has a

built-in function for sort that performs really well using only one single line. Whereas C doesn't and we conclude that the participants would rather lose performance coding a simpler algorithm than to write a complex one that could rival that of Python's Timsort. The H2 runtime which is shown on figure 9 would prove that C is really faster.

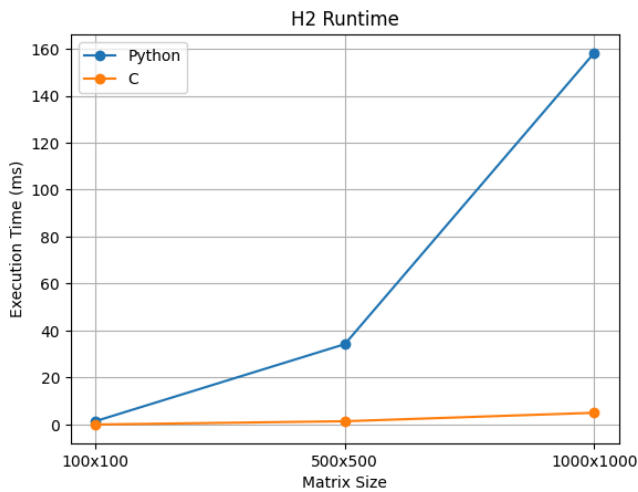


Fig. 9: H2 Runtime

As expected, C beat Python, marginally. The gap may not seem to differ but seeing the gap widened from 500 x 500 to 1000 x 1000 matrices, it shows. In a more direct comparison with a similar worst case and algorithm, C beats Python. We believe the cause may be that at only 10k elements, Python memory indirection overhead is still good but as we go to 1m elements it becomes very severe, the Python interpreter must box and unbox millions of integer objects, bound check array access and manage reference counts.

Another thing is Python's line by line interpretation and dynamic typing, the thing that made Python superior when compared to C in development. May be the downfall since it could accumulate over 1 million iterations of the innermost loop where C code just compiles to branch free vector instructions that the CPU executes at full speed.

This finding aligns very well with the fundamental goal of each language. Python being high level, prioritizes fast development. C being low level, prioritizes execution efficiency. This tradeoff is apparent when seeing the participants did the tasks faster with less lines in Python but the codes when comparing between the same algorithm gave way to C's faster execution speed.

That being said, when should we use Python and when should we use C? The answer is simple, for rapid prototyping, data science and analysis, and teams prioritizing time to market over performance the answer is Python but when tackling systems programming, real time application, computing intensive algorithms, and embedded systems that have resource limitations the answer is C.

Several limitations should also be considered when interpreting these results, primarily that all participants were undergraduate computer science students, which may limit generalization to professional developers. Secondly the tasks we cover are only algorithmic problems with varying difficulties. This result may not generalize well with web development where I/O is way more important than CPU, machine learning where libraries dominate, or real time

systems in which execution speed is not as important as latency.

Lastly, for researchers, future works should measure both development time and execution time simultaneously unlike prior works that isolate execution time and development time [14], [17], [19]. It should also include energy consumption and memory consumption to showcase how hard Python works the machine. It should also control the variables like algorithm differences when comparing language and try to avoid making the same mistake as us by having Timsort compared to Bubble Sort. Another thing is to recruit larger, bigger scope of participants, people who have mastered each respectable language. And lastly we believe this research could be taken further by implementing more variety in the algorithms that include string processing, graph algorithms, and much more. And also not only comparing C and Python but include Java to the mix and many other languages.

V. CONCLUSION

In this research, an apparently simple question is explored, but which cannot be answered in an empirical manner: Which programming language C or Python performs better in equal situations on the same tasks and by what magnitude? The findings confirm long-established theories: Python boosts the speed of writing a code, while C boosts the speed of running it. This is the first study to measure both variables in a scientific way. Python was shown to be more efficient in writing. Using 56% less lines of code and 1.6 times less minutes per task (on average - 3.8 minutes as opposed to 6.1 minutes). As for execution, C turned out to be much faster in executing code, 31 times faster for large-scale matrix transposition, and Python had advantages because of its better built-in algorithms. Moreover, Merge sort and insertion sort written in Python would be faster than naive sorting algorithms written in C. Although these results are typical, the novelty of this study lies in its ability to evaluate both development and execution dimensions at the same time, using real programmers as volunteers.

Importantly, the trade-off emerges from design decisions made, rather than from the languages themselves; Python trades efficiency at run-time for developer-friendly design and quick iterations, and C trades programmer-friendliness for low-level hardware manipulation and fast execution. The correct answer will depend on the nature of the problem itself—the rate at which the code runs, the number of developers maintaining it, and the required latency. Our framework provides the foundation upon which such questions can be answered.

That said, the researchers note the limitations that were experienced, as well as the considerations future researchers will need to take into account if they hope to perform a similar analysis to ours more successfully. First of all, in terms of scope and parameters, in hindsight, the sheer number of problems used— nine problems, each at three different difficulty levels—might have been beyond what we could manage with our resources and time. Future researchers should think about scope restrictions, and fewer, well-defined problems with clear constraints will provide more meaningful data. Better to have answered one question accurately than three questions poorly.

Concerning volunteer recruitment: this is one of the most difficult aspects of this process. Finding volunteers who had real-hands on experience with both C and Python programming languages and were willing to spend their time solving all nine tasks on those languages was not easy at all. In case other researchers decide to conduct a similar study, volunteer recruitment should be part of the project schedule from the very beginning; besides, they should take into account that some attrition will inevitably take place and think about whether the small number of well-selected participants would provide better results than the large number of heterogeneous ones.

VI. REFERENCES

- [1] A. Ritonga, "Studi Literatur Perbandingan Bahasa Pemrograman C++ Dan Bahasa Pemrograman Python Pada Algoritma Pemrograman," *Jurnal Teknik Informatika dan Teknologi Informatika*, pp. 56-63, 2023.
- [2] Y. He, "A Comparative Study of the Significance of Different Programming Languages," *Academic Journal of Computing & Information Science*, pp. 45-50, 2025.
- [3] A. Kumar, "Performance comparison of instrument automation pipelines using different programming languages," *Scientific Reports*, pp. 1-14, 2023.
- [4] V. Stoici, "An Empirical Study on the Performance and Energy Usage of Compiled Python Code," *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, pp. 45-56, 2025.
- [5] P. Jain, "Comparative Review of Python and C: Choosing the Right Language for Beginners," *International Journal of Engineering Trends and Applications*, vol. 11, no. 3, pp. 148-156, 2024.
- [6] R. Kesuma, *Algoritma Dan Programming Konsep Dasar, Logika, dan Implementasi dengan C++ & Python*, Payakumbuh: Serasi Media, 2025.
- [7] R. Kumar, "Python Versus C Language: A Comparison," *International Journal of Trend in Research and Development*, pp. 171-173, 2022.
- [8] A. Gujar, "C vs Python: A Cursory Look with Industry Opinion," *International Journal For Research*, pp. 55-64, 2023.
- [9] S. Ali, "A Pragmatic Comparison of Four Different Programming Languages," *ScienceOpen Preprints*, pp. 1-13, 2021.
- [10] A. Azenkouk, "The Influence of Programming Languages on Computational Efficiency and Performance," *International Journal of Research and Innovation in Social Science*, pp. 285-288, 2025.
- [11] P. Nanfack, "Analysis of programming languages used in solving energy problems," *Environmental Science, Computer Science, Engineering*, pp. 1-6, 2022.
- [12] Y. Zhang, "Comparative study of the execution efficiency of Python and C++ based on topological sorting," *International Conference on Machine Learning and Automation*, pp. 13-17, 2024.
- [13] L. Miller, "Comparative Study of Python and C++ in High-Performance AI Model Deployment," *International Journal of Computer Science*, pp. 1-5, 2025.
- [14] H. Patel, "The Enduring Relevance of C Programming: Analysing Its Efficiency, Portability and Role in Modern Computing," *A Global Journal of Interdisciplinary Studies*, pp. 546-550, 2025.
- [15] M. J. I. Antonio, "A Discerning Comparison between C++ and Python in Game Development," pp. 1-14, 2024.
- [16] B. R. Maheshwari, "Comparative Analysis of Program Execution Time Required by Python, R and Julia Compiler," *Indian Journal of Computer Science and Technology*, pp. 25-27, 2024.
- [17] Y. Tao, "Beyond Functional Correctness: An Exploratory Study on the Time Efficiency of Programming Assignments," *International Conference on Software Engineering: Software Engineering Education and Training*, pp. 320-330, 2024.
- [18] R. Pereira, "Ranking programming languages by energy efficiency," *Science of Computer Programming*, vol. 205, no. 102609, pp. 1-30, 2021.
- [19] P. Rysak, "Comparative analysis of C and Python on the basis of the execution time of applications implementing selected algorithms," *Journal Computer Science Institute*, vol. 26, pp. 93-99, 2023.
- [20] H. Kishorbhai, "THE ENDURING RELEVANCE OF C PROGRAMMING: ANALYSING ITS EFFICIENCY, PORTABILITY, AND ROLE IN MODERN COMPUTING," *A Global Journal of Interdisciplinary Studies*, pp. 546-550, 2025.
- [21] V. M. Ionescu, "Investigating the performance of MicroPython and C on ESP32 and STM32 microcontrollers," *International Symposium for Design and Technology in Electronic Packaging*, vol. 10, pp. 234-237, 2020.
- [22] A. Upadhya, "A comparative study on usage of Python and C/C++ for firmware development," *International Journal of Advance Research, Ideas and Innovation in Technology*, vol. 6, no. 3, pp. 557--559, 2020.
- [23] V. Cutting, "Comparative Review of JAVA and Python," *International Journal of Research and Development in Applied Science and Engineering*, vol. 21, no. 1, pp. 1-6, 2021.
- [24] M. Martignano, "Static Analysis for ADA, C/C++ and Python: Different Languages, Different Needs," *Joint publication with Ada User Journal*, vol. 42, no. 3, pp. 77-80, 2021.
- [25] U. Schmitt, "sympy2c: From symbolic expressions to fast C/C++ functions and ODE," *Astronomy and Computing*, vol. 42, pp. 1-11, 2023.
- [26] K. A. Onyango, "Comparative Analysis on the Evaluation of the Complexity of C, C++, Java, PHP and Python Programming Languages based on Halstead Software," *International Journal of Computer and Information Technology*, vol. 12, no. 1, pp. 8-19, 2023.
- [27] J. O. Ogala, "COMPARATIVE ANALYSIS OF C, C++, C# AND JAVA PROGRAMMING

- LANGUAGES," *Global Scientific Journal*, vol. 8, no. 5, pp. 1900-1913, 2020.
- [28] M. Shoaib, "A Comparative Study of Contemporary Programming Languages in Implementation of Classical Algorithms," *Journal of Information & Communication Technology*, vol. 14, no. 1, pp. 23-31, 2021.
- [29] A. Kaur, "A Comparative Study of Static Code Analysis tools for Vulnerability Detection in C/C++ and JAVA Source Code," *ScienceDirect*, pp. 1-7, 2023.
- [30] S. Sundaramoorthy, "A systematic analysis on performance and computational complexity of sorting algorithms," *Discover Computing*, pp. 1-30, 2025.
- [31] phuuun, "C-vs-Python-Study," 6 June 2026. [Online]. Available: <https://github.com/phuuun/C-vs-Python-Study>. [Accessed 26 June 2026].